INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Playwright Assertions Cheat Sheet

The web-first assertions you will use every day.

Introduction

Playwright’s web-first assertions auto-wait and retry, which removes most flakiness. This cheat sheet lists the assertions you will reach for most.

Element State

Assertion	Checks
toBeVisible()	Element is visible
toBeHidden()	Element is hidden/absent
toBeEnabled() / toBeDisabled()	Interactable state
toBeChecked()	Checkbox/radio checked
toBeFocused()	Element has focus

Content

Assertion	Checks
toHaveText()	Exact text
toContainText()	Partial text
toHaveValue()	Input value
toHaveAttribute()	Attribute value
toHaveCount()	Number of matched elements

Page

- await expect(page).toHaveURL(...)

- await expect(page).toHaveTitle(...)

Worked Example

await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible(); — Playwright retries until it is visible or times out, so you do not need manual waits.

Practical Exercise

Add three different web-first assertions to one of your tests (state, content and page). Confirm they auto-wait by testing an element that appears after a short delay.

Best Practices

- Prefer web-first assertions (expect(locator)) over manual checks.
- Assert the outcome a user cares about.
- Avoid sleeping — let assertions wait.

Common Mistakes

- Using expect on a value you fetched manually instead of the locator.
- Adding page.waitForTimeout before assertions.
- Asserting implementation detail instead of user-visible outcome.

> **INSIDE STLC PRO TIP**
>
> Web-first assertions auto-retry until the condition is met or it times out. Lean on them and you eliminate the single biggest source of flaky tests: manual waits.